

Student Referral Sheet – Faculty of Computing (Final Examination Jan-June 2025)

Student ID :

Module Code :

L4 - Creating a Table

```
CREATE TABLE STUDENT
(
    studentId INTEGER PRIMARY KEY,
    sName VARCHAR(30) NOT NULL,
    nic CHAR(10) UNIQUE,
    gpa FLOAT,
    progId VARCHAR(10) DEFAULT 'IT',
CONSTRAINT student_prog_fk FOREIGN KEY (progId)
REFERENCES programs(id) ON DELETE SET DEFAULT ON
UPDATE CASCADE,
CONSTRAINT gpa_ck CHECK (gpa <= 4.0 ) )
```

Modifications to Tables

Ex : Adding a new column to a table
. **ALTER TABLE** student **ADD** age **INT**
Ex : Adding a new constraint to a column
. **ALTER TABLE** student **ADD CONSTRAINT** chk_age **CHECK** (age > 18)
Ex : removing a column from a table
ALTER TABLE student **DROP COLUMN** age
Ex : Deleting a table
. **DROP TABLE** Employee

Inserting a row

```
INSERT INTO student VALUES (1000,'Amal','123456789V', 3.2, 'BM')
INSERT INTO student(studentId, sName, nic) VALUES (1001, 'Nimali',
'234567890V')
```

Deleting a row

```
DELETE student WHERE studentId=1000
Updating a row
UPDATE student
SET gpa=2.8
WHERE studentId=1001
```

LIKE operator

```
SELECT Name
FROM student
WHERE Name Like 'A%'
```

DISTINCT operator

(duplicate)
SELECT DISTINCT progId
FROM student

BETWEEN operator

```
SELECT studentID
FROM student
WHERE gpa BETWEEN 3.7 AND 4.00
```

ORDER BY Clause

```
SELECT Name, gpa
FROM student
ORDER BY gpa DESC;
```

[INNER] Join

Ex : Find the names of students who follow programs offered by SLIIT
Select s.Name
From student s, program p
where s.pid=p.ProgId and offerBy='SLIIT'
Or
Select s.Name
From student s
INNER JOIN program p **on** s.pid=p.progId
Where offerBy='SLIIT'

IN operator

Ex : Find the students who has obtained a 'A'.
Select s.Name
From Student s
Where s.SID **IN** (**Select** SID
from Grades
Where Grade='A')

Aggregation

```
SELECT AVG(gpa), MIN(gpa), MAX(gpa) , SUM(salary), COUNT(sid)
FROM student;
```

Grouping

Ex: Count the number of students who has followed each module.
Select CID, Count(SID)
From Student
Group by CID

HAVING Clause

Ex: Find courses which is followed by more than two students
Select CID, Count(SID)
From course
Group by CID
Having count(SID)>2

LS – VIEWS

emp (eid, ename, age, salary) ,dept (did, dname, budget,mangerId),
works (eid, did, pct_time)
Create a view named dept_info that contains name of the department,
budget and manager's name

```
CREATE VIEW dept_info(dname,budget,manager)
AS
SELECT d.dname, d.budget, e.ename FROM emp e, dept d
WHERE e.eid=d.managerId
```

Querying and Deleting a view

Querying a view can be done similar to a table.
Ex: **SELECT * FROM** dept info
Ex: **SELECT** dname **FROM** dep_info **WHERE** budget> 500,000

Dropping a view

Ex: **DROPVIEW** dept info

Updating Views

```
UPDATE emp_info
SET tot_pct= 90
WHERE eid = 1000
```

Why view?

Advantages of Views

Security: Users can access only specific data through views, limiting unauthorized access.

Query Simplicity: Views combine data from multiple tables into one, making complex queries easier.

Disadvantages of Views

Performance: Complex views can slow down queries since the DBMS must process underlying tables.

Update Restrictions: Some views (especially complex ones) cannot be updated directly.

Programming in T-SQL (variables /if else/looping/while)

Declare variable → **DECLARE** @<variable name> <data type>

Ex: **DECLARE** @DName **VARCHAR**(20)

Assign a Value → **SET** @<variable name> =<value>

Ex: **SET** @DName = 'SESD'

IF statement

```
IF (SELECT count(eid) FROM emp) > 1000
BEGIN
    PRINT 'Inside the IF statement'
    PRINT 'There are lesser than 1000 employees '
END
ELSE
    PRINT 'There are more than 1000 employees ! '
```

While statement

BREAK-Causes an exit from the innermost WHILE loop. Any statements appearing after the END keyword, marking the end of the loop, are executed.

CONTINUE-Causes the WHILE loop to restart, ignoring any statements after the CONTINUE keyword.

Ex :**WHILE** @count<=100

```
BEGIN
    INSERT INTO Employees
    VALUES (@count,CONCAT('Employee',@count))
    SET @count=@count+1
END
```

Create a function that returns the number of employees in a given department.

```
create function getEmpCount (@did char(12)) returns int
as
begin
    declare @ecount int
    select @ecount=count(*)
    from works w
    where w.did=@did
    return @ecount
end
```

Calling the function created previously

```
declare @result int
exec @result=get_empCount 'Admin'
print @result
```

Triggers

A trigger is a special type of stored procedure that automatically takes effect when the data in a specified table is modified.

A trigger is invoked in response to a
DDL statement (CREATE, ALTER etc.) or
DML statement (INSERT, UPDATE, or DELETE statement). '

DML triggers

Syntax :
CREATE TRIGGER trigger_name
ON { table | view }
{
 { { **FOR** | **AFTER** | **INSTEAD OF** }
 { { **INSERT** } [,] { **UPDATE** } [,]
 { **DELETE** } }
AS
sql_statement [...n]
}}

Ex :
Create trigger account_audit_trigg

```
On Account
For Insert, update
As
Begin
    Declare @ano int
    Declare @balance float
    Select @ano=accountNo,@balance=balance from inserted
    Insert into accountAudit(@ano,@balance,getdate())
end
```

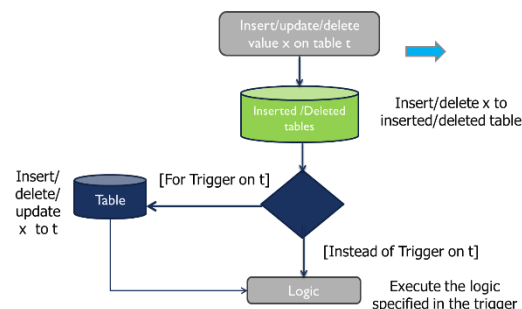
These events can be → **INSERT/UPDATE/DELETE**

Types of Triggers

BEFORE Trigger: Executes **before** the event occurs (used for validation).

AFTER Trigger: Executes **after** the event occurs (used for logging or backup).

INSTEAD OF Trigger: Replaces the action (commonly used with views).



I. Database Security (Lecture 08)

A. Aspects of Database Security

There are three main objectives when designing a secure database application:

- (1)**Secrecy:** Information should not be disclosed to unauthorized users².
- (2)**Integrity:** Only authorized users should be allowed to modify data³.
- (3)**Availability:** Authorized users should not be denied access⁴.

B. Access Control Mechanisms

A security policy defines who is authorized to do what. The two main mechanisms at the DBMS level to control access are:

- (1)**Discretionary Access Control (DAC):** Based on access rights (privileges) and mechanisms for granting them.
- (2)**Mandatory Access Control (MAC)**

C. Security in SQL Server

The SQL Server security framework manages access through

Authentication and Authorization

Authentication: Verifying the identities of users trying to connect to a SQL Server instance

Authorization: Determines which data resources authorized users can access and what actions they can take

This is achieved using **Principals**, **Securables**, and **Permissions**.

Principals : Individuals, groups, or processes granted access to the SQL Server instance, either at the server level or database level.

→ Server-level principals include logins and server roles.

→ Database-level principals include users and database roles.

Securable: Objects that make up the server and database environment.

The objects can be broken into three hierarchical levels:

→ Server-level securables include such objects as databases and availability groups.

→ Database-level securables include such objects as schemas and full-text catalogs.

→ Schema-level securables include such objects as tables, views, functions, and stored procedures.

Permissions: The types of access permitted to principals on specific securables. You can grant or deny permissions to securables at the server, database, or schema level.

Creating a login using windows authentication

```
CREATE LOGIN [domain_name\login_name]
FROM WINDOWS
[ WITH DEFAULT_DATABASE = database_name
| DEFAULT_LANGUAGE = language_name ];
```

Example **CREATE LOGIN** [win10b\winuser01] **FROM WINDOWS**
WITH DEFAULT_DATABASE = master, **DEFAULT_LANGUAGE** = us_english;

Creating a login using sql server

Syntax

CREATE LOGIN [Login Name] -- This is the User that you use for login
WITH PASSWORD = 'provide_password' **MUST_CHANGE**,
CHECK_EXPIRATION = ON, **CHECK_POLICY** = ON, **DEFAULT_DATABASE** = [Database Name],
DEFAULT_LANGUAGE = [Language Name];-- This is Optional

Example

```
CREATE LOGIN sqluser01
WITH PASSWORD = 'tempPW@56789'
MUST_CHANGE, CHECK_EXPIRATION = ON,
DEFAULT_DATABASE = master, DEFAULT_LANGUAGE = us_english;
```

